TP2 : Création d'une blockchain simple avec Python

Matière : Blockchain
Environnement : Python 3, VS Code, PyCharm ou Jupyter Notebook

Objectifs pédagogiques

À la fin de ce TP, l'étudiant sera capable de :

- Comprendre le fonctionnement d'une fonction de hachage cryptographique.
- Utiliser SHA-256 avec Python.
- Représenter un bloc sous forme d'objet Python.
- Construire une chaîne de plusieurs blocs.
- Comprendre le rôle du hash précédent dans le chaînage.
- Implémenter une fonction de vérification de l'intégrité d'une blockchain.
- Expérimenter les conséquences d'une modification des données.

Partie I — Rappel-cours : de la théorie à la programmation

1. Rappel : comment fonctionne une blockchain ?

Dans le TP1, nous avons découvert qu'une blockchain est un registre numérique partagé dans lequel les informations sont regroupées dans des blocs reliés entre eux.

Prenons l'exemple de trois transactions :

- Alice envoie 10 € à Bob.
- Bob envoie 5 € à Charlie.
- Charlie envoie 2 € à Alice.

Dans une blockchain simplifiée, ces transactions peuvent être enregistrées dans trois blocs successifs.

Bloc 1

- Transaction : Alice → Bob : 10 €
- Hash précédent : 0
- Hash actuel : A123

Bloc 2

- Transaction : Bob → Charlie : 5 €

- Hash précédent : A123
- Hash actuel : B456

Bloc 3

- Transaction : Charlie → Alice : 2 €
- Hash précédent : B456
- Hash actuel : C789

Les valeurs A123, B456 et C789 sont des exemples fictifs.

Chaque bloc contient une référence cryptographique au bloc précédent. Cette relation permet de construire une chaîne et de détecter certaines modifications de son historique.

Dans ce TP, nous allons programmer cette structure avec Python.

2. Comprendre le hachage cryptographique

2.1. Qu'est-ce qu'une fonction de hachage ?

Une fonction de hachage est un algorithme qui transforme une donnée de taille quelconque en une empreinte numérique de taille fixe.

Par exemple, nous pouvons calculer une empreinte à partir du texte :

Bonjour

Si nous modifions ce texte en :

bonjour

nous obtenons une empreinte différente.

Une fonction de hachage cryptographique possède plusieurs propriétés importantes :

- **Déterminisme :** une même donnée produit toujours le même hash avec le même algorithme.
- **Sensibilité aux modifications :** une petite modification peut produire un hash très différent.
- **Résistance à la préimage :** il est extrêmement difficile de retrouver une donnée à partir de son hash.
- **Résistance aux collisions :** il est extrêmement difficile de trouver deux données différentes produisant le même hash.

2.2. Pourquoi utiliser le hachage dans une blockchain ?

Le hachage joue trois rôles principaux.

Identifier le contenu d'un bloc

Chaque bloc possède une empreinte calculée à partir de ses informations.

Détecter les modifications

Si une transaction change, le hash recalculé change généralement.

Relier les blocs

Chaque bloc conserve le hash du bloc précédent. Cette dépendance permet de vérifier la cohérence de la chaîne.

2.3. La fonction SHA-256

SHA-256 est une fonction de hachage cryptographique produisant une empreinte de 256 bits, généralement représentée par 64 caractères hexadécimaux.

Python fournit la bibliothèque hashlib, qui permet d'utiliser SHA-256 sans installation supplémentaire.

Exemple :

```
import hashlib


message = "Bonjour"


hash_message = hashlib.sha256(
   message.encode("utf-8")
).hexdigest()


print(hash_message)
```

Dans ce programme :

- hashlib fournit les fonctions de hachage.
- encode("utf-8") convertit le texte en octets.
- sha256() calcule l'empreinte.
- hexdigest() retourne l'empreinte sous forme hexadécimale.

3. Représenter un bloc en Python

Un bloc peut être représenté par un objet contenant plusieurs attributs.

Dans notre blockchain simplifiée, chaque bloc contiendra :

Attribut	**Description**
index	Position du bloc dans la chaîne
transaction	Information enregistrée dans le bloc
previous_hash	Hash du bloc précédent
hash	Empreinte du bloc actuel

Nous utiliserons une classe Python appelée Bloc.

Une classe permet de définir une structure commune pour créer plusieurs objets.

Exemple :

```
class Bloc:


  def __init__(self, index, transaction, previous_hash):


    self.index = index

    self.transaction = transaction

    self.previous_hash = previous_hash
```

La méthode __init__() est appelée automatiquement lorsqu'un nouvel objet est créé.

Le mot-clé self permet d'accéder aux attributs de l'objet.

Pour créer un bloc :

```
bloc1 = Bloc(
  1,
  "Alice envoie 10 euros a Bob",
  "0"
)
```

Ce bloc possède un identifiant, une transaction et une référence au bloc précédent.

4. Calculer le hash d'un bloc

Pour calculer le hash d'un bloc, nous devons utiliser les informations qu'il contient.

Dans notre modèle, nous utiliserons :

- l'identifiant du bloc ;
- la transaction ;
- le hash du bloc précédent.

Ces informations seront regroupées dans une chaîne de caractères avant d'être transmises à SHA-256.

Exemple de méthode :

```
def calculer_hash(self):


  contenu = (
    str(self.index)
    + "|"
    + self.transaction
    + "|"
    + self.previous_hash
  )


  return hashlib.sha256(
    contenu.encode("utf-8")
  ).hexdigest()
```

L'utilisation du séparateur | permet de distinguer les différentes informations dans la représentation du bloc.

Le hash obtenu dépend ainsi du contenu du bloc et de sa référence au bloc précédent.

5. Représenter une blockchain avec une liste Python

Une blockchain simplifiée peut être représentée par une liste contenant plusieurs objets de type Bloc.

Exemple :

```
blockchain = [bloc1, bloc2, bloc3]
```

Les blocs sont conservés dans leur ordre de création.

Pour ajouter un nouveau bloc, nous devons :

1. Récupérer le dernier bloc de la chaîne.
2. Récupérer son hash.
3. Créer un nouveau bloc contenant ce hash comme référence précédente.
4. Ajouter le nouveau bloc à la liste.

Le premier bloc est particulier : il ne possède pas de bloc précédent.

Il est appelé **bloc Genesis**, ou bloc de genèse.

Dans notre implémentation simplifiée, nous utiliserons la valeur "0" pour représenter son hash précédent.

6. Vérifier l'intégrité d'une blockchain

La vérification de l'intégrité consiste à contrôler que les blocs n'ont pas été modifiés et que leurs liens restent cohérents.

Deux vérifications seront réalisées :

Vérification du contenu

Pour chaque bloc, nous recalculons son hash et nous le comparons au hash enregistré.

Vérification du chaînage

Pour chaque bloc, à l'exception du premier, nous vérifions que son champ previous_hash correspond au hash du bloc précédent.

Ces contrôles permettent de détecter des incohérences dans la chaîne.

Il faut cependant distinguer cette vérification locale d'une véritable validation par un réseau blockchain. Notre programme ne comporte pas encore de consensus, de signature numérique ou de mécanisme de preuve de travail.

Partie II — TP pratique guidé

Exercice 1 — Découvrir SHA-256

Étape 1 : préparation

Créez un dossier nommé :

TP2_Blockchain

Dans ce dossier, créez un fichier Python nommé :

exercice1.py

Étape 2 : calculer un hash

Écrivez le programme suivant :

```
import hashlib


texte = "Blockchain"


hash_texte = hashlib.sha256(
   texte.encode("utf-8")
).hexdigest()


print("Texte :", texte)
print("Hash :", hash_texte)
```

Exécutez le programme.

Étape 3 : expérimenter

Effectuez les modifications suivantes :

1. Remplacez Blockchain par blockchain.
2. Exécutez à nouveau le programme.
3. Remplacez le texte par Blockchain et cryptographie.
4. Exécutez le programme.
5. Comparez les différentes empreintes obtenues.

Travail demandé

Répondez aux questions suivantes :

1. Les différentes entrées produisent-elles le même hash ?
2. Quelle est la longueur d'un hash SHA-256 en caractères hexadécimaux ?
3. La longueur du texte influence-t-elle la longueur du hash ?
4. Pourquoi le hachage est-il utile dans une blockchain ?
5. Peut-on facilement retrouver le texte original à partir du hash ? Justifiez.

Exercice 2 — Créer son premier bloc

Étape 1 : créer le fichier

Créez un fichier nommé :

exercice2.py

Étape 2 : définir la classe Bloc

Écrivez le programme suivant :

```
import hashlib


class Bloc:


  def __init__(self, index, transaction, previous_hash):


    self.index = index

    self.transaction = transaction

    self.previous_hash = previous_hash


    self.hash = self.calculer_hash()


  def calculer_hash(self):


    contenu = (

      str(self.index)

      + "|"

      + self.transaction

      + "|"

      + self.previous_hash

    )
```

```
        return hashlib.sha256(
            contenu.encode("utf-8")
        ).hexdigest()
```

Étape 3 : créer un objet Bloc

Ajoutez le code suivant :

```
bloc1 = Bloc(
    1,
    "Alice envoie 10 euros a Bob",
    "0"
)


print("Index :", bloc1.index)
print("Transaction :", bloc1.transaction)
print("Hash precedent :", bloc1.previous_hash)
print("Hash actuel :", bloc1.hash)
```

Travail demandé

1. Exécutez le programme.
2. Identifiez les attributs de la classe Bloc.
3. Identifiez la méthode responsable du calcul du hash.
4. Modifiez la transaction en remplaçant 10 euros par 100 euros.
5. Exécutez à nouveau le programme.
6. Comparez les empreintes obtenues.

Questions

1. Quel est le rôle de la méthode __init__() ?
2. À quoi sert l'instruction self.hash = self.calculer_hash() ?
3. Pourquoi l'identifiant est-il converti en chaîne de caractères ?
4. Quels attributs interviennent dans le calcul du hash ?

5. Que remarquez-vous après la modification de la transaction ?

Exercice 3 — Construire une blockchain de trois blocs

Étape 1 : créer le fichier

Créez un fichier nommé :

exercice3.py

Reprenez la classe Bloc développée dans l'exercice précédent.

Étape 2 : créer les blocs

Ajoutez le code suivant :

```
bloc1 = Bloc(
  1,
  "Alice envoie 10 euros a Bob",
  "0"
)

bloc2 = Bloc(
  2,
  "Bob envoie 5 euros a Charlie",
  bloc1.hash
)

bloc3 = Bloc(
  3,
  "Charlie envoie 2 euros a Alice",
  bloc2.hash
)
```

Étape 3 : créer la blockchain

Ajoutez :

blockchain = [bloc1, bloc2, bloc3]

Étape 4 : afficher les blocs

Ajoutez le programme suivant :

for bloc in blockchain:

```
    print("----------------------")
    print("Bloc :", bloc.index)
    print("Transaction :", bloc.transaction)
    print("Hash precedent :", bloc.previous_hash)
    print("Hash actuel :", bloc.hash)
```

Travail demandé

1. Exécutez le programme.
2. Observez les informations affichées pour chaque bloc.
3. Comparez le hash du Bloc 1 avec le hash précédent du Bloc 2.
4. Comparez le hash du Bloc 2 avec le hash précédent du Bloc 3.
5. Modifiez la transaction du Bloc 2 dans son constructeur.
6. Exécutez à nouveau le programme et observez les modifications.

Questions

1. Comment les blocs sont-ils reliés entre eux ?
2. Pourquoi le Bloc 2 utilise-t-il bloc1.hash ?
3. Pourquoi le Bloc 3 utilise-t-il bloc2.hash ?
4. Quel est le rôle de la liste blockchain ?
5. Que se passe-t-il si nous remplaçons bloc1.hash par "12345" lors de la création du Bloc 2 ?

Exercice 4 — Vérifier l'intégrité de la blockchain

Étape 1 : créer une fonction de vérification

Reprenez le programme de l'exercice 3.

Ajoutez la fonction suivante :

```
def verifier_blockchain(blockchain):

    for i in range(len(blockchain)):

        bloc = blockchain[i]

        # Verification du hash du bloc actuel
        if bloc.hash != bloc.calculer_hash():
            return False

        # Verification du chainage
        if i > 0:

            bloc_precedent = blockchain[i - 1]

            if bloc.previous_hash != bloc_precedent.hash:
                return False

    return True
```

Étape 2 : vérifier la blockchain

Ajoutez :

```
print(
    "Blockchain valide :",
    verifier_blockchain(blockchain)
)
```

Exécutez le programme et observez le résultat.

Étape 3 : modifier une transaction

Ajoutez les instructions suivantes :

```
bloc1.transaction = "Alice envoie 1000 euros a Bob"


print(
  "Blockchain valide apres modification :",
  verifier_blockchain(blockchain)
)
```

Exécutez à nouveau le programme.

Travail demandé

1. Comparez les résultats avant et après la modification.
2. Identifiez la condition qui détecte la modification.
3. Expliquez le rôle de la fonction calculer_hash() dans cette vérification.
4. Identifiez la condition qui vérifie le lien entre deux blocs.
5. Expliquez pourquoi la fonction retourne False dans certaines situations.

Questions

1. Pourquoi doit-on recalculer le hash pour vérifier un bloc ?
2. Pourquoi la vérification du hash précédent est-elle nécessaire ?
3. Quelle est la différence entre vérifier le contenu d'un bloc et vérifier son chaînage ?
4. Cette fonction suffit-elle à garantir la sécurité d'une véritable blockchain ? Justifiez.

Partie III — Exercices d'application

Exercice 5 — Ajouter un quatrième bloc

À partir du programme développé précédemment, ajoutez un quatrième bloc contenant la transaction suivante :

David envoie 15 euros à Alice.

Le nouveau bloc doit posséder :

- L'identifiant 4.

- La transaction correspondante.
- Le hash du Bloc 3 comme hash précédent.
- Son propre hash calculé automatiquement.

Travail demandé

1. Créez l'objet bloc4.
2. Ajoutez-le à la liste blockchain.
3. Affichez les informations des quatre blocs.
4. Vérifiez l'intégrité de la nouvelle chaîne.

Questions

1. Quel hash doit être utilisé comme hash précédent du Bloc 4 ?
2. Pourquoi ne faut-il pas utiliser le hash du Bloc 1 ?
3. Que se passe-t-il si le Bloc 4 contient un hash précédent incorrect ?

Exercice 6 — Expérimenter les modifications

Pour chaque expérience, repartez d'une blockchain valide.

Réalisez les manipulations suivantes :

Expérience	Manipulation
1	Modifier la transaction du Bloc 2 sans recalculer son hash.
2	Modifier le hash précédent du Bloc 3.
3	Ajouter correctement un cinquième bloc.
4	Modifier la transaction du Bloc 1 et recalculer uniquement son hash.
5	Modifier la transaction du Bloc 1, puis recalculer les hash de tous les blocs successifs en mettant à jour leurs références.

Pour chaque expérience :

1. Exécutez la fonction verifier_blockchain().
2. Notez le résultat obtenu.
3. Identifiez les informations modifiées.
4. Expliquez pourquoi la vérification réussit ou échoue.

Complétez le tableau suivant :

Expérience	Résultat de la vérification	Explication
1		
2		
3		
4		
5		

Partie IV — Questions de compréhension

Répondez individuellement aux questions suivantes.

1. Qu'est-ce qu'une fonction de hachage cryptographique ?
2. Quel est le rôle de SHA-256 dans notre programme ?
3. Quelle est la différence entre une transaction et un hash ?
4. Pourquoi chaque bloc contient-il le hash du bloc précédent ?
5. Qu'est-ce que le bloc Genesis ?
6. Pourquoi utilisons-nous une classe Python pour représenter un bloc ?
7. Pourquoi utilisons-nous une liste Python pour représenter la blockchain ?
8. Quel est le rôle de la fonction verifier_blockchain() ?
9. Que se passe-t-il lorsqu'une transaction est modifiée sans recalculer son hash ?
10. Un attaquant peut-il recalculer les hash de plusieurs blocs après une modification ?
11. Pourquoi notre programme ne constitue-t-il pas encore une blockchain décentralisée ?
12. Quelles fonctionnalités faudrait-il ajouter pour se rapprocher d'une véritable blockchain ?

Partie V — Synthèse

Complétez le tableau suivant avec vos propres mots.

Notion	Définition ou explication	Exemple
Fonction de hachage		
SHA-256		
Bloc		
Transaction		
Hash précédent		
Bloc Genesis		
Chaînage		
Intégrité des données		
Vérification d'une blockchain		

Travail à rendre

Chaque étudiant doit remettre :

1. Les fichiers Python des exercices réalisés.
2. Le programme final contenant une blockchain d'au moins quatre blocs.
3. La fonction de vérification de l'intégrité.
4. Le tableau complété de l'exercice 6.
5. Les réponses aux questions de compréhension.